

Binary Search

On a correctly sorted array, compare at the midpoint and discard the impossible half until the key is found or the range is empty.

This complete offline chapter connects ideas, diagrams, exact traces, commented Java and exam practice. Switch themes for reading; printing always uses the light presentation and includes every required trace and answer.

GUIDE 2026-10-07.1 · STANDALONE HTML AND PDF SHARE THE SAME CONTENT

1. Meet the array and the goal

An array stores values at zero-based indexes. A search key is the value you want; the answer is its position or a failure marker.

8	18	23	29	34	42	54	63	72	85
0	1	2	3	4	5	6	7	8	9

Index labels belong to positions, not to the values. Find key 18.

The last valid index is 9. Returning 18 would confuse a value with an index.

2. Definition and plain-language idea

Binary search is a comparison search that repeatedly halves a sorted candidate range.

8	18	23	29	34	42	54	63	72	85
0 low	1	2	3	4 mid	5	6	7	8	9 high

The first midpoint divides a sorted inclusive range into smaller candidates.

On a correctly sorted array, compare at the midpoint and discard the impossible half until the key is found or the range is empty.



Since key 18 is smaller than 34, retain only indexes 0..3; the next midpoint is index 1.

Precondition: ordering must match the comparison direction and searched field. Running this code on unsorted input can incorrectly report that a present key is absent.

3. Vocabulary before the trace

Term	Meaning in this chapter
Key	The target value.
low / high	Inclusive candidate boundaries.
mid	$\text{low} + (\text{high} - \text{low}) / 2$ with integer division.
Probe	One visited midpoint; not necessarily one Java Boolean evaluation.
Empty range	$\text{low} > \text{high}$.
Invariant	If the key exists, a matching position remains within $\text{low}..\text{high}$.

4. Visual trace: predict, then advance

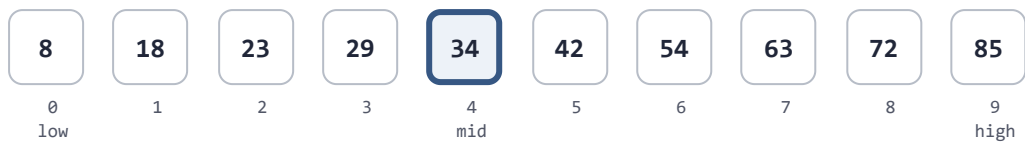
Before pressing Next, say which position will be inspected and what the bounds/index will become. The marked row and picture share one selected step.

Search for 18

▼ Full trace table - select a row to jump to its scene

Step	low	high	mid	Value	Evaluated conditions and update
1	0	9	4	34	$34 == 18 \rightarrow \text{false}$; $34 < 18 \rightarrow \text{false}$; $\text{high} = 3$
2	0	3	1	18	$18 == 18 \rightarrow \text{true}$; return 1

Probe 1: index 4



Key 18; inclusive candidate range 0..9.

$34 == 18 \rightarrow \text{false}$; $34 < 18 \rightarrow \text{false}$; $\text{high} = 3$

Probe 2: index 1



Key 18; inclusive candidate range 0..3.

$18 == 18 \rightarrow \text{true}$; return 1

Return 1 after 2 probes.

5. Why the movement is valid

In ascending order, if $\text{arr}[\text{mid}]$ is smaller than the key, every value to its left is also too small. Exclude the midpoint itself with $\text{low} = \text{mid} + 1$. The symmetric case uses $\text{high} = \text{mid} - 1$.

Less than key
discard



mid checked
exclude



Possible larger values
retain

Each unsuccessful probe must strictly shrink the inclusive range.

A one-cell range still needs a comparison, so use $\text{low} \leq \text{high}$, not $\text{low} < \text{high}$.

6. Fully commented Java implementation

JAVA

```
public static int binarySearch(int[] arr, int key) {
    // PRECONDITION: arr is in ascending order by the searched value.
    int low = 0, high = arr.length - 1;
    // Inclusive endpoints: a one-element range must still be checked.
    while (low <= high) {
        // Overflow-safe midpoint; integer division rounds down here.
        int mid = low + (high - low) / 2;
        if (arr[mid] == key) return mid;
        if (arr[mid] < key) {
            // Mid was already checked; exclude it from the next range.
            low = mid + 1;
        } else {
            // All values from mid rightwards are too large.
            high = mid - 1;
        }
    }
    // low > high means no candidate index remains.
    return -1;
}
```

Build it from scratch

- State the sorted-input precondition.
- Initialize the inclusive range.
- Repeat while a candidate remains.
- Calculate the midpoint; test equality.
- Exclude mid and the impossible side.
- Return -1 after the range becomes empty.

7. Recursive form and the call stack

JAVA

```
public static int binarySearchRec(int[] arr, int key, int low, int high) {
    // Empty range must return before calculating/accessing mid.
    if (low > high) return -1;
    int mid = low + (high - low) / 2;
    if (arr[mid] == key) return mid;
    // Exactly one recursive child is needed; the other half is impossible.
    if (key < arr[mid]) return binarySearchRec(arr, key, low, mid - 1);
    return binarySearchRec(arr, key, mid + 1, high);
    // First call: binarySearchRec(arr, key, 0, arr.length - 1).
}
```

CALL STACK - NEWEST FRAME AT THE TOP

binarySearchRec(0, 3)

mid 1 matches 18: return 1

binarySearchRec(0, 9)

Waiting: Return the result from child range 0..3

The matching child returns 1; its parent forwards that result. Only one branch is called.

Initial call: `binarySearchRec(arr, key, 0, arr.length - 1)`. Depth is $O(\log n)$ because each child retains at most about half the range.

8. Complete runnable program

Save as `BinaryDemo.java`, compile with `javac`, then run the class name without an extension.

JAVA

```
import java.util.Arrays;

public class BinaryDemo {
    public static int binarySearch(int[] arr, int key) {
        // PRECONDITION: arr is in ascending order by the searched value.
        int low = 0, high = arr.length - 1;
        // Inclusive endpoints: a one-element range must still be checked.
        while (low <= high) {
            // Overflow-safe midpoint; integer division rounds down here.
            int mid = low + (high - low) / 2;
            if (arr[mid] == key) return mid;
            if (arr[mid] < key) {
                // Mid was already checked; exclude it from the next range.
                low = mid + 1;
            } else {
                // All values from mid rightwards are too large.
                high = mid - 1;
            }
        }
        // low > high means no candidate index remains.
        return -1;
    }

    public static void main(String[] args) {
        int[] values = {8, 18, 23, 29, 34, 42, 54, 63, 72, 85};
        System.out.println(binarySearch(values, 18));
    }
}
```

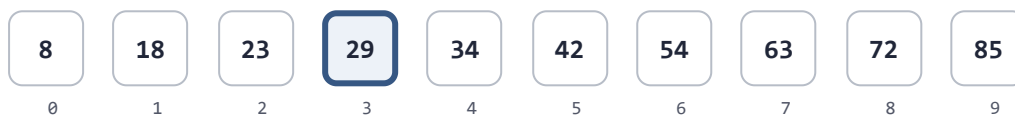
EXPECTED OUTPUT

1

9. Five worked cases and exact trace tables

1. Absent between values

Input [8, 18, 23, 29, 34, 42, 54, 63, 72, 85]; key 31.



Final probe/check at index 3; return -1.

Probe	low	high	mid	Value	Evaluated conditions and update
1	0	9	4	34	$34 == 31 \rightarrow \text{false}$; $34 < 31 \rightarrow \text{false}$; high = 3
2	0	3	1	18	$18 == 31 \rightarrow \text{false}$; $18 < 31 \rightarrow \text{true}$; low = 2
3	2	3	2	23	$23 == 31 \rightarrow \text{false}$; $23 < 31 \rightarrow \text{true}$; low = 3
4	3	3	3	29	$29 == 31 \rightarrow \text{false}$; $29 < 31 \rightarrow \text{true}$; low = 4

Result -1; 4 midpoint probes.

2. First value

Input [8, 18, 23, 29, 34, 42, 54, 63, 72, 85]; key 8.



Final probe/check at index 0; return 0.

Probe	low	high	mid	Value	Evaluated conditions and update
1	0	9	4	34	$34 == 8 \rightarrow \text{false}$; $34 < 8 \rightarrow \text{false}$; high = 3

Probe	low	high	mid	Value	Evaluated conditions and update
2	0	3	1	18	$18 == 8 \rightarrow \text{false}$; $18 < 8 \rightarrow \text{false}$; $\text{high} = 0$
3	0	0	0	8	$8 == 8 \rightarrow \text{true}$; return 0

Result 0; 3 midpoint probes.

3. Duplicate key

Input [10, 20, 20, 30]; key 20.

10

20

20

20

30

0

1

2

3

4

Final probe/check at index 2; return 2.

Probe	low	high	mid	Value	Evaluated conditions and update
1	0	4	2	20	$20 == 20 \rightarrow \text{true}$; return 2

Result 2; 1 midpoint probes.

4. Descending order

Input [85, 72, 63, 54, 42, 34, 29, 23, 18, 8]; key 18; descending comparisons.

85

72

63

54

42

34

29

23

18

8

0

1

2

3

4

5

6

7

8

9

Final probe/check at index 8; return 8.

Probe	low	high	mid	Value	Evaluated conditions and update
1	0	9	4	42	$42 == 18 \rightarrow \text{false}$; $42 > 18 \rightarrow \text{true}$; $\text{low} = 5$
2	5	9	7	23	$23 == 18 \rightarrow \text{false}$; $23 > 18 \rightarrow \text{true}$; $\text{low} = 8$
3	8	9	8	18	$18 == 18 \rightarrow \text{true}$; return 8

Result 8; 3 midpoint probes.

5. Empty and singleton

Input [18]; key 31.

18

0

Final probe/check at index 0; return -1.

Probe	low	high	mid	Value	Evaluated conditions and update
1	0	0	0	18	18 == 31 → false; 18 < 31 → true; low = 1

Result -1; 1 midpoint probes. For [] the loop/body is never entered and -1 is returned with zero probes.

10. Variations: order, duplicates and reference values

For descending arrays, equality is unchanged. If key is larger than arr[mid], search LEFT by reducing high; otherwise search RIGHT by increasing low.

A standard binary search returns whichever equal midpoint it encounters first. To find the first duplicate, record the match and continue left instead of returning immediately.

10

20

20

20

30

01234

first midpoint

Searching for 20 returns index 2, even though the first occurrence is index 1.

11. Properties and why the cost occurs

Property	Result	Reason
Best time	O(1)	First midpoint is a match.
Average / worst	O(log n)	Every failed probe removes approximately half the candidates.

Property	Result	Reason
Extra space	O(1) iterative; O(log n) recursive	Bounds versus suspended frames.
Input order	Required	Discard decisions depend on order.
Duplicates	Any matching midpoint	No first/last guarantee.
Input mutated?	No	Only bounds change.

n candidates

One midpoint probe

n / 2

One midpoint probe

n / 4

One midpoint probe

At most 1

Match or empty range

Halving repeatedly gives logarithmic growth. A probe means one visited midpoint, even if Java evaluates two comparison expressions.

A successful or unsuccessful midpoint visit is counted as one probe in these tables. The exact Java code can evaluate equality and then a direction comparison on a failed probe; do not call probe count the exact number of all Boolean expressions.

12. When to choose this method

Choose binary search for repeated lookups on data already sorted by the relevant key. Sorting solely for one lookup may cost more than a linear scan. Binary search also relies on efficient random access; a linked list does not provide array-like midpoint access.

13. Common mistakes and repairs

Mistake	Consequence	Repair
Search an unsorted array	Can discard a present match.	Sort by the same searched key/direction first.
Use mid instead of mid + 1 or mid - 1	Range may stop shrinking.	Exclude the already-tested midpoint.
Use low < high	Misses a final single candidate.	Use low <= high.
Return a value instead of index	Wrong output contract.	Return mid.
Assume first duplicate	Standard search can return a later occurrence.	Continue left after recording a match.

14. Exam-style practice with explanations

A. Trace key 31 on the main input. Show every index/bound and the failure result. [6]

Probe	low	high	mid	Value	Evaluated conditions and update
1	0	9	4	34	$34 == 31 \rightarrow \text{false}$; $34 < 31 \rightarrow \text{false}$; $\text{high} = 3$
2	0	3	1	18	$18 == 31 \rightarrow \text{false}$; $18 < 31 \rightarrow \text{true}$; $\text{low} = 2$
3	2	3	2	23	$23 == 31 \rightarrow \text{false}$; $23 < 31 \rightarrow \text{true}$; $\text{low} = 3$
4	3	3	3	29	$29 == 31 \rightarrow \text{false}$; $29 < 31 \rightarrow \text{true}$; $\text{low} = 4$

Every retained candidate is eliminated; return -1. Four midpoint probes are required.

B. Explain the best and worst time using input examples. [4]

Best: key equals the first midpoint, one probe. Worst: key absent or only located after the range repeatedly halves, logarithmic probes.

C. Debug the bounds and return statement. [4]

low and high are inclusive. Use $\text{low} \leq \text{high}$, calculate the overflow-safe midpoint, and exclude mid using ± 1 . The not-found return belongs after the range empties.

D. Return the first occurrence of a duplicate. What must change? [3]

Keep a result variable. On equality record mid, set $\text{high} = \text{mid} - 1$, and continue. Return the recorded index after the loop.

E. Empty array: can this code access index 0? [2]

No. high is -1, so $\text{low} \leq \text{high}$ is false. It returns -1 without reading a cell.

15. Test-ready summary and mastery check

On a correctly sorted array, compare at the midpoint and discard the impossible half until the key is found or the range is empty.

- State the sorted-order precondition.
- Show low, high, mid and evaluated conditions on each probe.
- Shrink the range with $\text{mid} \pm 1$.
- Separate probe counts from all Java comparison evaluations.
- Distinguish iterative space from recursive stack space.

CSI247 visual study guide · Binary Search · 2026-10-07.1

Structural reference: the author-supplied CSI247 guide, with topic-specific adaptation and repo-owned teaching content. All traces in this artifact are generated by deterministic simulations of its stated procedures. Counts distinguish key comparisons from loop tests, probes, swaps, shifts and array writes.